15. Context: Cancellation and Deadlines

15. Context: Cancellation and Deadlines

Phase: Phase 2 — Core language depth
Estimated time: 1–2 days
Slug: `15-context`

Learning objectives

- Pass context.Context as the first parameter
- Derive cancel and timeout contexts
- Propagate cancellation through goroutines and RPCs
- Never store contexts in structs long-term (usually)

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Why context?

Distributed and concurrent programs need a signal: “stop working, the caller left”.

```
ctx, cancel := context.WithTimeout(parent, 2*time.Second)
defer cancel()

select {
case <-ctx.Done():
    return ctx.Err()
case res := <-results:
    return res, nil
}
```

Rules of thumb

1. `ctx` is the first argument to functions that do I/O or wait.
2. Call `cancel` to free resources (use defer).
3. Do not pass `nil` context — use `context.Background()` or `TODO()` at edges.
4. Values in context are for request-scoped data (trace IDs), not optional parameters.

Cancellation is cooperative: your code must check `ctx.Done()` or call APIs that do.

Common mistakes (especially from Python instincts)

- Starting goroutines without tying them to ctx
- Using context for mandatory function parameters
- Forgetting defer cancel()

Exercises

Work in `exercises/15-context/`. Read that folder’s `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/15-context/` until you have an honest failing attempt.

Self-check

1. What does WithTimeout return?
2. Why is cancel required?
3. What error do you get when timed out?

Further reading

- https://go.dev/blog/context
- https://pkg.go.dev/context